

Marcus Cepeda

Synchrony Skills Academy

20 May 2026

Cybersecurity Basics 2

Implement and Explain Advanced Cybersecurity Defense Strategies:

Step 1 - Run the existing Kali Linux VM

Step 2 - Create an Ubuntu Server VM

Step 3 - Make sure both VMs are running and on the same network

Step 4 - On the Ubuntu server VM, find its IP address by typing:

```
ip a
```

Step 5 - Write the Ubuntu Server IP address down (Example: 192. 168. 56. 102)

Assume that the Kali Linux IP address is 192. 168. 56. 101 from here forward.

Step 6 - Update the Ubuntu Server VM by running:

```
sudo apt update && sudo apt upgrade -y
```

Step 7 - Update the Kali Linux VM by running:

```
sudo apt update && sudo apt full-upgrade -y
```

Step 8 - Create a non-root admin account on the Ubuntu Server VM by running:

```
sudo adduser secadmin
```

```
sudo usermod -aG sudo secadmin
```

Step 9 - Generate an SSH key on the Kali Linux VM by running:

```
ssh-keygen -t ed25519 -C "secadmin@lab"
```

When asked for a file location, press Enter to use the default.

When asked for a passphrase, set one.

Step 10 - On the Kali Linux VM, copy the public key to the Ubuntu server by running:

```
ssh-copy-id secadmin@192.168.56.102
```

Step 11 - Test SSH access by running the following on the Kali Linux VM:

```
ssh secadmin@192.168.56.102
```

You should be able to login using the key.

Then, exit the session:

```
exit
```

Step 12 - Create the first Zero Trust layer, which is firewall restrictions:

```
sudo apt install ufw -y
```

```
sudo ufw default deny incoming
```

```
sudo ufw default allow outgoing
```

```
sudo ufw allow from 192.168.56.101 to any port 22 proto tcp
```

```
sudo ufw enable
```

```
sudo ufw status verbose
```

Step 13 - Create the second Zero Trust layer, which is SSH key-only authentication:

On the Ubuntu Server VM, edit the SSH config:

```
sudo nano /etc/ssh/sshd_config
```

Set/change these lines:

```
PasswordAuthentication no
```

```
PubkeyAuthentication yes
```

```
PermitRootLogin no
```

Save and exit:

```
Ctrl + O / Enter / Ctrl + X
```

Restart SSH:

```
sudo systemctl restart ssh
```

Step 14 - Make sure that the access controls actually work:

From the Kali Linux VM, connect normally:

```
ssh secadmin@192.168.56.102
```

It should work with the key.

Now try a wrong-user OR password-only attempt from the Kali Linux VM:

```
ssh wronguser@192.168.56.102
```

It should fail.

Step 15 - Implement the third layer of defense, `fail2ban`, on the Ubuntu Server VM:

```
sudo apt install fail2ban -y
```

```
sudo systemctl enable fail2ban
```

```
sudo systemctl start fail2ban
```

```
sudo systemctl status fail2ban
```

Step 16 - Describe the 3 layers of Defense in Depth that were used:

Layer 1: Network Security

- UFW firewall
- Only allow SSH from the admin workstation
- Deny all other inbound traffic

Layer 2: Host Security

- Non-root admin account
- SSH key-only login

- Root login disabled
- Automatic updates enabled

Layer 3: Monitoring & Detection

- Logging enabled
 - `fail2ban` installed to block repeated brute-force attempts
 - Reviewing `/var/log/auth.log` by running: `sudo tail -f /var/log/auth.log`
 - Monitor for unauthorized login attempts
-

Step 17 - Demonstrate Supply Chain Security with one example:

Example: Software verification before installation

The risk is:

- malicious or tampered software
- fake packages
- compromised download mirrors

The mitigation is:

- use official sources only
- verify signatures or hashes before installing
- do not install untrusted binaries

How to apply Supply Chain Security in this example:

1. Download the vendor's checksum file and signature file.
2. Import the vendor's public key.
3. Verify the signature on the checksum file and the SHA256 hash matches.
4. Install only if both checks pass.

Step 18 - Describe one advanced security model:

The Bell-LaPadula model has two core rules:

- No read up: a lower clearance user cannot read higher-classified data
- No write down: a higher clearance user cannot write data to a lower classification level

The Bell-LaPadula model was used to protect confidentiality by restricting information based on classification levels. Users with lower clearance couldn't read higher-classified data, and users with higher clearance weren't able to write sensitive data into lower-security locations.

Implement Incident Response and Handling:

Step 1 - Create a 5-step incident response framework:

Preparation

- Before an incident occurs, logging should be enabled, backups available, evidence storage prepared, and response tools ready.

Identification

- The incident is identified when suspicious activity is observed, such as repeated failed SSH logins, invalid usernames, or unusual log entries. A person would confirm the event by reviewing `/var/log/auth.log` on the Ubuntu server.

Containment

- The affected system is isolated to stop further unauthorized access. Containment is performed by blocking the source IP with UFW so the attacker can't connect to SSH.

Eradication

- The cause of the incident is removed. Eradication includes confirming no unauthorized accounts or keys were added, and ensuring that the fail2ban/firewall rules are still active.

Recovery

- The system is returned to normal operation after verification. Recovery includes restoring allowed access for any legitimate accounts, confirming that SSH works normally, and checking that the logs don't show any more suspicious activity.

Step 2 - Create incidents:

On the Kali Linux VM, run:

```
mkdir -p
```

```
~/Cybersec_Basics_2/IRP/{evidence, notes, screenshots, chain_of  
_custody}
```

Also run:

```
date -u
```

Write down the UTC time and use UTC for all evidence records.

On the Kali Linux VM, make a few failed SSH attempts to the Ubuntu server:

```
ssh fakeuser@192.168.56.102
```

If prompted, do not complete a login. Do this a few times with different fake usernames.

Example:

```
ssh admin99@192.168.56.102
```

```
ssh test123@192.168.56.102
```

Step 3 - Confirm incidents:

On the Ubuntu Server VM, run:

```
sudo grep -E "Failed password|Invalid user|authentication failure|Accepted publickey" /var/log/auth.log | tail -n 30
```

Also run:

```
sudo journalctl -u ssh --no-pager | tail -n 30
```

Take screenshots of both outputs.

Step 4 - Use 1 forensic tool for data collection:

On the Ubuntu Server VM, run:

```
sudo dd if=/var/log/auth.log of=/home/secadmin/auth.log.dd  
bs=4M conv=noerror,sync status=progress
```

Next, hash the collected file:

```
sha256sum /home/secadmin/auth.log.dd | tee  
/home/secadmin/auth.log.dd.sha256
```

Then, copy the collected evidence to the Kali Linux VM:

```
scp /home/secadmin/auth.log.dd  
secadmin@192.168.56.101: ~/Cybersec_Basics_2/IRP/evidence/  
scp /home/secadmin/auth.log.dd.sha256  
secadmin@192.168.56.101: ~/Cybersec_Basics_2/IRP/evidence/
```


Step 5 - Ensure that collection and documentation are supported by at least 2 forms of evidence:

Both evidence types:

1. Log file evidence

- /var/log/auth.log
- copied as: auth.log.dd
- hashed with SHA-256

2. Screenshot evidence

- screenshot of failed SSH attempts
- screenshot of grep output from auth.log
- screenshot of ufw status numbered

Step 6 - Document chain of custody:

Evidence ID	Description	Source	Collected By	Date/Time (UTC)	Hash	Storage Location
E-01	auth.log forensic image	Ubuntu server	Your name	2026-05-22 13:30 UTC	SHA-256 value	IRP/evidence/auth.log.dd
E-02	Screenshot of failed SSH attempts	Kali terminal	Your name	2026-05-22 13:32 UTC	N/A	IRP/screenshots/failed-ssh.png
E-03	Screenshot of log review	Ubuntu terminal	Your name	2026-05-22 13:35 UTC	N/A	IRP/screenshots/auth-log-review.png

Step 7 - Show incident triage and prioritization by categorizing 3 types of incidents:

Incident Type	Severity	Business Impact	Priority
Confirmed ransomware on a server	Critical	Service outage, data encryption, and possible data loss	Immediate
Unauthorized SSH brute-force attempts	High	Risk of unauthorized access and compromise	High
Single phishing email reported before clicking	Medium	Low immediate impact but could lead to compromise	Normal / Monitor It

Step 8 - Write a post-incident analysis:

The incident involved repeated unauthorized SSH login attempts against the Ubuntu server from the Kali host. The event was identified through `/var/log/auth.log` and confirmed by repeated failed authentication entries. Containment was successful after the source IP was blocked with UFW. No unauthorized account creation, successful login, or evidence of persistence was found. The system was returned to a normal state after verification of the SSH configuration and a review of logs.

Lessons Learned:

- Strong authentication and firewall restrictions significantly reduce the risk of unauthorized access.
- Logging and rapid review of authentication events are very important for early detection and containment.

Demonstrate SOC (Security Operations Center) Fundamentals:

Step 1 - Identify at least 3 primary SOC roles and their responsibilities:

Tier 1 SOC Analyst

- Monitors alerts, reviews dashboards and logs, performs initial triage, and escalates any suspicious events.

Tier 2 SOC Analyst

- Investigates the alerts in more detail, correlates log sources, confirms whether the event is real, and helps contain the threat.

SOC Engineer/SIEM Administrator

- Maintains the monitoring tools, alerts, log sources, dashboards, and sensor configuration.
-

Step 2 - Configure 1 monitoring tool:

Install Suricata and other supporting tools on the Kali Linux VM:

```
sudo apt update
```

```
sudo apt install suricata suricata-update nmap -y
```

Find your network range:

```
ip a
```

```
ip route
```

Identify:

- The Kali Linux VM IP
- The Ubuntu Server IP
- The subnet (Example: 192.168.56.0/24)

Open Suricata's main config:

```
sudo nano /etc/suricata/suricata.yaml
```

Find HOME_NET and set it to your subnet, for example:

```
HOME_NET: "[192.168.56.0/24]"
```

Save and exit:

```
Ctrl + O / Enter / Ctrl + X
```

Open the local rules file:

```
sudo nano /etc/suricata/rules/local.rules
```

Add these two rules:

```
alert icmp any any -> $HOME_NET any [msg: "LAB ALERT - ICMP  
Echo Request"; itype: 8; sid: 1000001; rev: 1; ]
```

```
alert tcp any any -> $HOME_NET 22 [msg: "LAB ALERT - SSH Port  
22 Connection Attempt"; flags: S; sid: 1000002; rev: 1; ]
```

Save and exit:

```
Ctrl + O / Enter / Ctrl + X
```

Test the Suricata configuration:

```
sudo suricata -T -c /etc/suricata/suricata.yaml
```

If the test passes, continue.

Start Suricata on your network interface:

```
sudo suricata -c /etc/suricata/suricata.yaml -i eth0 -D
```

Verify Suricata is running:

```
ps -ef | grep suricata
```

Watch the alert log:

```
sudo tail -f /var/log/suricata/fast.log
```

Keep this open while you generate traffic.

Step 3 - Generate 2 different security alerts:

1. ICMP Ping Alert

On the Kali Linux VM, ping the Ubuntu server:

```
ping -c 4 192.168.56.102
```

This should trigger the ICMP rule and create an alert in `fast.log`.

1. SSH Connection Attempt Alert

Use nmap to scan port 22:

```
nmap -sS -p 22 192.168.56.102
```

The scan should trigger the second alert rule.

Step 4 - Collect Evidence:

Take screenshots of:

1. `suricata -T` test passing
2. `fast.log` showing the ICMP alert
3. `fast.log` showing the SSH/port 22 alert
4. `nmap/ping` command output
5. `ps -ef | grep suricata` showing the sensor is running

Also, keep these text logs as evidence:

- /var/log/suricata/fast.log
- /var/log/suricata/eve.json

Useful commands to review evidence:

```
sudo grep "LAB ALERT" /var/log/suricata/fast.log
```

```
sudo grep ' "event_type": "alert"' /var/log/suricata/eve.json  
| tail -n 20
```

Step 5 - Show evidence of how 2 security alerts were generated, investigated, and resolved:

Alert 1: ICMP echo request

How it was generated:

- A ping test from Kali to the Ubuntu server generated ICMP traffic.

How it was investigated:

- The alert was reviewed in Suricata's fast.log and eve.json. The analyst checked the source IP, destination IP, timestamp, and alert message.

How it was resolved:

- The alert was confirmed as an expected lab connectivity test and was closed as low severity.

Alert 2: SSH port 22 connection attempt

How it was generated:

- A scan or SSH attempt from Kali to the Ubuntu server triggered the port 22 rule.

How it was investigated:

- The Suricata alert was reviewed, the source and destination were identified, and there was confirmation that the traffic was targeting SSH.

How it was resolved:

- The event was treated as suspicious reconnaissance. It was resolved by documenting the event, confirming no unauthorized login occurred, and, if desired, blocking the source IP on Ubuntu with UFW:

```
sudo ufw deny from 192.168.56.101 to any port 22 proto  
tcp  
  
sudo ufw status numbered
```

Step 6 - Analyze 1 identified threat and show how it was detected using SOC tools:

Threat identified: Reconnaissance activity against SSH port 22

Detection method: Suricata detected a TCP SYN attempt to port 22 and generated a rule-based alert. I correlated the source IP, destination IP, and packet type in Suricata logs to determine that the event was a scan or connection attempt aimed at a remote access service.

Develop and Implement Security Policies and Governance:

Step 1 - Security Policy:

A. Access Control Policy

Access to systems and data must follow the principle of least privilege. Users are granted only the permissions necessary to complete approved job duties. Administrative access must be limited to authorized personnel only. Shared accounts are prohibited unless formally approved and documented. Strong authentication must be used for privileged access, and root login should be disabled when possible. Access rights must be reviewed regularly and removed when no longer required.

B. Data Protection Policy

Sensitive data must be protected from unauthorized viewing, modification, or disclosure. Data should be stored in restricted locations with proper file permissions and should not be transferred to unapproved devices or services. Backups must be maintained for important files, and data classification should be used to separate public, internal, and confidential information. Any sensitive information must be handled according to its classification level.

C. System Use Policy

Systems and network resources must be used for authorized business or educational purposes only. Users may not install unauthorized software, disable security controls, or attempt to bypass access restrictions. Passwords and authentication keys must not be shared. Suspicious activity, malware, or policy violations must be reported immediately. Systems must be kept updated and monitored for security events.

Step 2 - Governance Structure:

System Owner

- Responsible for approving the security policy, defining the overall security requirements, and deciding what data and systems require protection.

IT Administrator

- Responsible for enforcing access control, managing accounts, applying updates, reviewing logs, and maintaining technical safeguards.

End User

- Responsible for following the policy, protecting credentials, using systems appropriately, and reporting suspicious activity.

Security Analysts/SOC

- Responsible for monitoring alerts, reviewing logs, investigating policy violations, and escalating incidents when needed.

Management/Supervisors

- Responsible for ensuring the policy is communicated, training is completed, and violations are handled appropriately.

Step 3 - Compliance Requirements:

This policy aligns with the NIST Cybersecurity Framework by supporting core security functions such as Identify, Protect, Detect, Respond, and Recover. Access control supports the Protect function. Logging and monitoring support Detect. Incident handling supports Respond and Recover. Also, regularly reviewing permissions and enforcing policies supports governance and risk reduction.

Step 4 - Demonstrate Policy Implementation:

Create a policy document on the Ubuntu Server VM:

```
sudo mkdir -p /opt/security/policies
```

```
sudo nano /opt/security/policies/security_policy.txt
```

Paste the policy text from above into the file, then save and exit.

Make the policy readable to users:

```
sudo chmod 644 /opt/security/policies/security_policy.txt
```

```
ls -l /opt/security/policies/security_policy.txt
```

Add a login banner to communicate this policy to the users:

```
sudo nano /etc/issue.net
```

Paste:

```
Authorized use only. All activity may be monitored. Follow  
the security policy located in  
/opt/security/policies/security_policy.txt.
```

Enable the banner in SSH:

```
sudo nano /etc/ssh/sshd_config
```

Add/edit this line:

```
Banner /etc/issue.net
```

Restart SSH:

```
sudo systemctl restart ssh
```

For Evidence: take a screenshot when you log in from Kali Linux and see the banner.

Step 5 - Enforce the Security Policy:

Create a protected directory:

```
sudo mkdir -p /srv/confidential
```

```
sudo groupadd secgroup
```

```
sudo chown root:secgroup /srv/confidential
```

```
sudo chmod 750 /srv/confidential
```

Add authorized users to the group:

```
sudo usermod -aG secgroup secadmin
```

Create a sample confidential file:

```
sudo nano /srv/confidential/secret.txt
```

Put in sample text, then save & exit:

```
Confidential training data.
```

Now, restrict the file:

```
sudo chown root:secgroup /srv/confidential/secret.txt
```

```
sudo chmod 640 /srv/confidential/secret.txt
```

```
ls -l /srv/confidential
```

Produce Effective Security Documentation:

Step 1 - Create this directory structure on the Kali Linux VM:

```
mkdir -p
```

```
~/Cybersec_Basics_2/Documentation/{Procedures, Processes, Play  
books, Knowledge_Base}
```

Step 2 - Write a cybersecurity procedure document:

```
nano
```

```
~/Cybersec_Basics_2/Documentation/Procedures/SSH_Hardening_P  
rocedure.md
```

This will contain:

SSH Hardening Procedure

Purpose

This procedure describes how to implement SSH hardening controls on a Linux server to reduce the risk of unauthorized remote access.

Scope

This procedure applies to all Linux servers that provide SSH access.

Security Control

Secure Shell (SSH) Access Control

Procedure

Step 1: Install OpenSSH Server

Verify the OpenSSH server package is installed:

```
sudo apt install openssh-server -y
```

Step 2: Create a Dedicated Administrative User

```
sudo adduser secadmin  
sudo usermod -aG sudo secadmin
```

Step 3: Generate SSH Keys

On the administrative workstation:

```
ssh-keygen -t ed25519
```

Step 4: Copy Public Key to Server

```
ssh-copy-id secadmin@SERVER_IP
```

Step 5: Disable Password Authentication

Edit:

```
sudo nano /etc/ssh/sshd_config
```

Set:

```
PasswordAuthentication no  
PubkeyAuthentication yes
```

Step 6: Disable Root Login

```
PermitRootLogin no
```

Step 7: Restart SSH

```
sudo systemctl restart ssh
```

Step 8: Verify Access

Test login using:

```
ssh secadmin@SERVER_IP
```

Verification

Successful login using SSH keys confirms implementation.

Expected Outcome

Only authorized users with approved SSH keys can access the server remotely.

Step 3 - Demonstrate Process Documentation:

Create a step-by-step process for patch management:

```
nano
```

```
~/Cybersec_Basics_2/Documentation/Processes/Patch_Management  
_Process.md
```

This will contain:

Patch Management Process

Purpose

Ensure systems remain protected against known vulnerabilities through regular updates.

Frequency

Monthly or immediately for critical security updates.

Process

Step 1: Identify Available Updates

```
sudo apt update
```

Review available packages:

```
apt list --upgradable
```

Step 2: Assess Risk

Determine whether updates contain:

- Security fixes
- Bug fixes
- Feature updates

Prioritize security updates first.

Step 3: Create Backup

Create a system snapshot or backup before applying updates.

Step 4: Apply Updates

```
sudo apt upgrade -y
```

Step 5: Reboot if Required

```
sudo reboot
```

Step 6: Verify System Functionality

Confirm:

- Services are running
- Users can log in
- Applications function normally

Step 7: Document Completion

Record:

- Date
- Administrator
- Updates installed
- Verification results

Expected Outcome

Systems remain protected against known vulnerabilities and continue operating normally.

Step 4 - Write Security Playbook #1:

Create the document:

nano

~/Cybersec_Basics_2/Documentation/Playbooks/Unauthorized_SSH
_Access_Playbook.md

It will contain:

Security Playbook: Unauthorized SSH Access Attempt

Trigger

Repeated failed SSH login attempts detected.

Identification

1. Review SSH logs:

```
sudo grep "Failed" /var/log/auth.log
```

2. Identify source IP address.

Containment

Block source IP:

```
sudo ufw deny from ATTACKER_IP
```

Verify rule:

```
sudo ufw status
```

Eradication

1. Review user accounts:


```
cut -d: -f1 /etc/passwd
```

2. Verify no unauthorized SSH keys exist.

Recovery

1. Confirm legitimate access works.
2. Review logs for continued activity.

Documentation

Record:

- Source IP
 - Time detected
 - Actions taken
 - Outcome
-

Step 5 - Write Security Playbook #2:

Create the document:

```
nano  
~/Cybersec_Basics_2/Documentation/Playbooks/Malware_Infection_Playbook.md
```

It will contain:

Security Playbook: Suspected Malware Infection

Trigger

System exhibits suspicious behavior or malware alert is generated.

Identification

1. Review logs.
2. Identify affected host.
3. Determine scope of impact.

Containment

1. Disconnect affected system from network.
2. Prevent lateral movement.

Eradication

1. Remove malicious files.
2. Apply security updates.
3. Verify persistence mechanisms have been removed.

Recovery

1. Restore clean files from backup if necessary.
2. Reconnect system to network.
3. Monitor for recurrence.

Documentation

Record:

- Host name
 - Indicators observed
 - Containment actions
 - Recovery actions
 - Final status
-

Step 6 - Demonstrate Knowledge Base Management:

Create the knowledge base repository:

```
mkdir -p
```

```
~/Cybersec_Basics_2/Documentation/Knowledge_Base/{Linux_Security, Incident_Response, Network_Security}
```

Resource 1:

Create:

nano

~/Cybersec_Basics_2/Documentation/Knowledge_Base/Linux_Security/SSH_Best_Practices.md

Contents:

Category: Linux Security

Topics:

- SSH key authentication
- Disable root login
- Strong passphrases
- Least privilege
- Logging and monitoring

Resource 2:

Create:

nano

~/Cybersec_Basics_2/Documentation/Knowledge_Base/Incident_Response/Incident_Response_Reference.md

Contents:

Category: Incident Response

Topics:

- Preparation
- Identification
- Containment
- Eradication
- Recovery
- Evidence handling

Resource 3:

Create:

nano

~/Cybersec_Basics_2/Documentation/Knowledge_Base/Network_Security/Firewall_Reference.md

Contents:

Category: Network Security

Topics:

- UFW firewall basics
- Inbound filtering
- Outbound filtering
- Network segmentation
- Access control

Show Evidence of Completion:

```
sudo apt install tree -y
```

```
tree ~/Cybersec_Basics_2/Documentation
```

Take screenshots of this as well as all of the individual documents as evidence.